DEEP LEARNING FROM SCRATCH

A Guide for Self-Learners

Episode VI

Alive

You've done the math. Every gradient. Every equation.
Now it's time to watch it breathe.

Now it wakes up.

The hardest part is done. Now comes the reward.

Why this episode?

Every equation from Episode V becomes working code.
Every gradient becomes a line of Python.
By the end, you'll have a neural network that learns.

"Talk is cheap. Show me the code."
– Linus Torvalds

Pierre Chambet

March 2026

Part of the *Deep Learning From Scratch* series — an independent reconstruction of AI.

Contents

What You'll Learn in This Episode

After these 20 pages, you won't just understand neural networks in theory — you'll have *built one yourself*, from scratch, in Python.

1. How to **initialize** a neural network with random parameters
2. How to implement **forward propagation** in NumPy
3. How to turn gradient equations into **backpropagation code**
4. How to build a complete **training loop**
5. How to visualize what the network learns: the **decision boundary**
6. How to apply your network to **real images** (cats vs dogs)
7. Why your network will eventually **fail** — and what that means

1 The Reward

In the last episode, you did the hardest part.

You derived every gradient by hand. You traced the chain rule through two layers of neurons. You wrote down forward propagation, backpropagation, and the update rule.

All of that was preparation for this moment.

Where We Left Off

Episode V gave you the complete mathematical blueprint of a two-layer neural network:

- Forward propagation: $Z = Wx + b$, $A = \sigma(Z)$
- Backpropagation: dW, db for every layer
- Update rule: $W \leftarrow W - \alpha \cdot dW$

The theory is done. The equations are ready. Now we code.

Here's the truth about deep learning:

Once you understand the math, the code is **shockingly simple**.

Every function we'll write today is a direct translation of an equation you already know. No new theory. No new derivations. Just implementation.

Let's Build

Open the companion notebook. Fire up Python.
We're building a neural network from scratch.
Colab: https://colab.research.google.com/github/Pchambet/Deep-Learning-from-Scratch/blob/main/notebooks/06_alive.ipynb

2 The Dataset — A Problem Worth Solving

Before we build anything, we need a problem to solve.

Here's the scenario:

> You're a data scientist on an expedition. You've discovered a new region with unknown plant species. Some are safe to eat. Some are toxic. Your survival depends on classifying them correctly.

You measure two features for each plant: **leaf length** and **leaf width**.

You plot the data, and this is what you see:

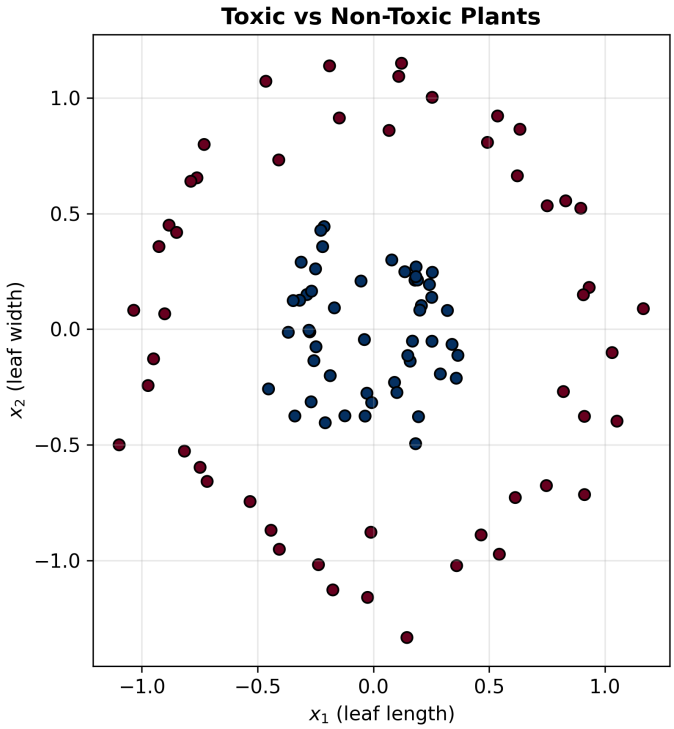


Figure 1: Two classes of data arranged in concentric circles — impossible to separate with a straight line. Generated from `make_circles` with `n_samples=100`, `noise=0.1`, `factor=0.3`.

The two colors represent the two classes: one forms an inner cluster, the other a ring around it. No straight line can separate them. A single neuron would fail here.

Why This Matters

This is exactly the kind of problem that requires a **neural network**. A single neuron draws a line. A network draws curves.
The data comes from `sklearn.datasets.make_circles` — a classic toy dataset for testing nonlinear classifiers.

In code, we generate it like this:

```
from sklearn.datasets import make_circles

X, y = make_circles(n_samples=100, noise=0.1, factor=0.3, random_state
    =0)
X = X.T                              # shape: (2, 100)
y = y.reshape((1, y.shape[0]))       # shape: (1, 100)
```

Convention: Data Shape

Throughout this episode, we follow this convention:

- X has shape `(n_features, n_samples)` — each column is one data point
- y has shape `(1, n_samples)` — one label per sample

This is the standard layout for vectorized neural network computation.

3 Initialization — Giving the Network a Body

Before the network can learn anything, it needs a **structure**.

We need to decide:

1. **How many inputs?** That's determined by the data — here, $n_0 = 2$.
2. **How many hidden neurons?** We choose this — let's start with $n_1 = 32$.
3. **How many outputs?** Binary classification means $n_2 = 1$.

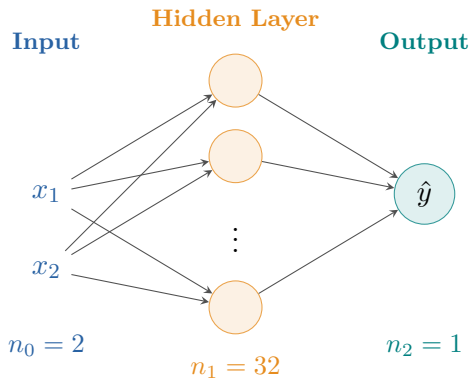


Figure 2: Architecture of our two-layer neural network: 2 inputs, 32 hidden neurons, 1 output.

Now we need to create the **parameters** for each layer:

- **Layer 1** (input → hidden): weight matrix W_1 of shape (n_1, n_0) and bias b_1 of shape $(n_1, 1)$
- **Layer 2** (hidden → output): weight matrix W_2 of shape (n_2, n_1) and bias b_2 of shape $(n_2, 1)$

Why Random Initialization?

We initialize weights **randomly** — not to zero.
If all weights started at zero, every neuron would compute the exact same thing. They'd all have the same gradients, the same updates, and they'd stay identical forever.
Random initialization **breaks the symmetry**: each neuron starts differently, learns differently, and becomes useful.

Here's the code:

```python
def initialization(n0, n1, n2):
    W1 = np.random.randn(n1, n0)    # (32, 2)
    b1 = np.random.randn(n1, 1)     # (32, 1)
    W2 = np.random.randn(n2, n1)    # (1, 32)
    b2 = np.random.randn(n2, 1)     # (1, 1)

    parameters = {
        'W1': W1, 'b1': b1,
        'W2': W2, 'b2': b2
    }
    return parameters
```

Checkpoint: Dimensions

Always check your matrix shapes. This is the number one source of bugs in neural network code.

Parameter	Shape	Why
W_1	(n_1, n_0)	n_1 neurons, each with n_0 weights
b_1	$(n_1, 1)$	one bias per neuron in layer 1
W_2	(n_2, n_1)	n_2 outputs, each connected to n_1 neurons
b_2	$(n_2, 1)$	one bias per output neuron

That's it. Four matrices. The network now has a body.
It doesn't know anything yet — but it *exists*.

4 Forward Propagation — The First Breath

Now we send data through the network for the first time.

This is **forward propagation**: the process of computing the output of each layer from input to output, one layer at a time.

From Episode V

Forward propagation for a two-layer network:
Note: Episode V used `tanh` *in the hidden layer. In this episode, we use sigmoid in both layers for implementation simplicity; the forward/backprop structure stays the same.*
Layer 1:

$$Z_1 = W_1 \cdot X + b_1 \qquad A_1 = \sigma(Z_1)$$

Layer 2 (output):

$$Z_2 = W_2 \cdot A_1 + b_2 \qquad A_2 = \sigma(Z_2)$$

A_2 is the network's prediction.

In Python, this becomes:

```python
def forward_propagation(X, parameters):
    W1 = parameters['W1']
    b1 = parameters['b1']
```

```
    W2 = parameters['W2']
    b2 = parameters['b2']

    # Layer 1
    Z1 = W1.dot(X) + b1             # Linear transformation
    A1 = 1 / (1 + np.exp(-Z1))      # Sigmoid activation

    # Layer 2 (output)
    Z2 = W2.dot(A1) + b2            # Linear transformation
    A2 = 1 / (1 + np.exp(-Z2))      # Sigmoid activation

    activations = {'A1': A1, 'A2': A2}
    return activations
```

That's it.

Six lines of actual computation. Same architecture as Episode V — now alive.

Tracing the Dimensions

Let's follow the shapes through the network step by step:

Operation	Shape	Explanation
X	$(2, 100)$	2 features, 100 samples
$W_1 \cdot X$	$(32, 2) \times (2, 100) = (32, 100)$	
Z_1	$(32, 100)$	32 neurons, 100 samples
$A_1 = \sigma(Z_1)$	$(32, 100)$	activations of hidden layer
$W_2 \cdot A_1$	$(1, 32) \times (32, 100) = (1, 100)$	
$A_2 = \sigma(Z_2)$	$(1, 100)$	one prediction per sample

If any shape doesn't match, NumPy will tell you immediately.

The network just took its first breath.

It processed 100 data points through 32 neurons, passed through two layers of activation, and produced 100 predictions — all in a fraction of a second.

The predictions are garbage right now (random weights, remember?).

But that's about to change.

5 Backpropagation — Learning from Mistakes

This is the moment where everything comes together.

In Episode V, you derived the gradients of the loss with respect to every parameter. You spent pages computing $\frac{\partial \mathcal{L}}{\partial W_2}$, $\frac{\partial \mathcal{L}}{\partial b_2}$, $\frac{\partial \mathcal{L}}{\partial W_1}$, $\frac{\partial \mathcal{L}}{\partial b_1}$.

Let's see what all that math looks like in code.

5.1 The Cost Function

First, we need to measure how wrong the network is.

We use the same **log-loss** (binary cross-entropy) as always:

$$\mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(A_2^{(i)}) + (1-y^{(i)})\log(1-A_2^{(i)})\right]$$

In code:

```python
def log_loss(A, y):
    m = y.shape[1]
    epsilon = 1e-15    # prevent log(0)
    return -1/m * np.sum(
        y * np.log(A + epsilon) +
        (1 - y) * np.log(1 - A + epsilon)
    )
```

5.2 The Gradient Equations

Now, the gradients. Here they are — side by side with their code. Episode V used `tanh` in the hidden layer, while here we use sigmoid: the backprop pattern is identical, only the hidden activation derivative changes.

Episode V Equations → Python Code

Output layer gradients:

$$dZ_2 = A_2 - y \qquad dW_2 = \frac{1}{m}\,dZ_2 \cdot A_1^\top \qquad db_2 = \frac{1}{m}\sum dZ_2$$

Hidden layer gradients:

$$dZ_1 = W_2^\top \cdot dZ_2 \circ A_1 \circ (1 - A_1) \qquad dW_1 = \frac{1}{m}\,dZ_1 \cdot X^\top \qquad db_1 = \frac{1}{m}\sum dZ_1$$

And the code:

```python
def back_propagation(X, y, parameters, activations):
    A1 = activations['A1']
    A2 = activations['A2']
    W2 = parameters['W2']
    m = y.shape[1]

    # Output layer
    dZ2 = A2 - y
    dW2 = 1/m * dZ2.dot(A1.T)
    db2 = 1/m * np.sum(dZ2, axis=1, keepdims=True)

    # Hidden layer
    dZ1 = np.dot(W2.T, dZ2) * A1 * (1 - A1)
    dW1 = 1/m * dZ1.dot(X.T)
    db1 = 1/m * np.sum(dZ1, axis=1, keepdims=True)

    gradients = {
        'dW1': dW1, 'db1': db1,
        'dW2': dW2, 'db2': db2
```

```
    }
    return gradients
```

Count the lines.
Six lines of gradient computation. That's it.
This is why you did the math.

The Hadamard Product

Notice the `*` operator in the dZ_1 line. This is the element-wise product (Hadamard product), denoted $\circ$ in math.
It's **not** a matrix multiplication — it's component-by-component.
`np.dot()` is matrix multiplication. `*` is element-wise.
Getting this wrong is a classic bug.

Let's take a moment to appreciate what just happened.
In Episode V, we spent several pages deriving dZ_1 through the chain rule. We went from $\frac{\partial \mathcal{L}}{\partial A_2}$ to $\frac{\partial \mathcal{L}}{\partial Z_2}$ to $\frac{\partial \mathcal{L}}{\partial A_1}$ to $\frac{\partial \mathcal{L}}{\partial Z_1}$.
All of that — **one line of code**:

```
dZ1 = np.dot(W2.T, dZ2) * A1 * (1 - A1)
```

Theory → Code

This is the pattern of deep learning engineering:

1. Derive the math **once** (Episode V)
2. Translate it into code **once** (this episode)
3. Never touch it again — frameworks will do it for you

But you need to understand it. **That's what makes you dangerous.**

6 Update and Prediction — Closing the Loop

Two more functions and we're done building.

6.1 Update — Four Subtractions

Now that we have the gradients, we update the parameters.
This is **gradient descent**: move each parameter in the direction that reduces the loss.

$$W \leftarrow W - \alpha \cdot dW \qquad b \leftarrow b - \alpha \cdot db$$

where α is the **learning rate** — how big each step is.

```
def update(gradients, parameters, learning_rate):
    W1 = parameters['W1'] - learning_rate * gradients['dW1']
    b1 = parameters['b1'] - learning_rate * gradients['db1']
```

```
    W2 = parameters['W2'] - learning_rate * gradients['dW2']
    b2 = parameters['b2'] - learning_rate * gradients['db2']

    parameters = {
        'W1': W1, 'b1': b1,
        'W2': W2, 'b2': b2
    }
    return parameters
```

Four subtractions. That's learning.

The Learning Rate

Too high and the network overshoots, oscillating wildly around the minimum. Too low and it barely moves, taking forever to converge.
There's no formula to pick the right α. You try, you observe, you adjust.
That's part of the craft.

6.2 Prediction — Making Decisions

The network outputs a continuous value $A_2 \in [0, 1]$ — a probability.

To turn it into a decision, we apply a **threshold**:

$$\hat{y} = \begin{cases} 1 & \text{if } A_2 \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

```
def predict(X, parameters):
    activations = forward_propagation(X, parameters)
    A2 = activations['A2']
    return A2 >= 0.5
```

We compute accuracy by comparing predictions to true labels:

```
from sklearn.metrics import accuracy_score

y_pred = predict(X, parameters)
accuracy = accuracy_score(y.flatten(), y_pred.flatten())
```

Why 0.5?

At $A_2 = 0.5$, the network is maximally uncertain — it has no preference between class 0 and class 1. Above 0.5, it leans toward class 1. Below, class 0.
This threshold is a design choice. In some applications (e.g., medical diagnosis), you might lower it to catch more positives — even at the cost of more false alarms.

Checkpoint — Your Toolkit So Far

Before going further, let's take stock. You have written **seven functions** from scratch, each handling one piece of the puzzle:

#	Function	Role
1	`initialization`	Create W and b with random values
2	`forward_propagation`	$X \to Z_1 \to A_1 \to Z_2 \to A_2$
3	`log_loss`	Measure how wrong the network is
4	`back_propagation`	Compute all gradients
5	`update`	Adjust parameters via gradient descent
6	`predict`	Turn probabilities into decisions
7	`neural_network`	Orchestrate everything (next section)

Nothing here was imported from a library. Every line is yours.

7 The Training Loop — Putting It All Together

We have all the pieces. Let's assemble them.

The training loop repeats the same four steps over and over:

1. **Forward propagation** — compute predictions
2. **Cost** — measure the error
3. **Backpropagation** — compute gradients
4. **Update** — adjust parameters

Each iteration is called an **epoch**.

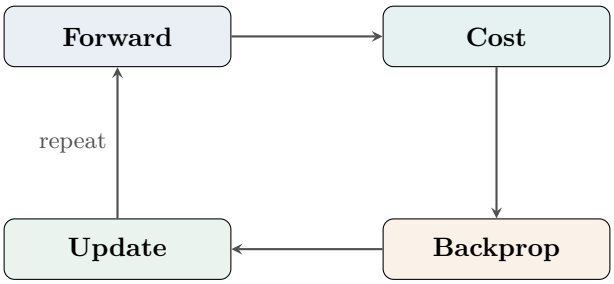


Figure 3: The training loop: four steps, repeated for each epoch.

```
def neural_network(X, y, n1=32, learning_rate=0.1, n_epochs=1000):
    # Dimensions
    n0 = X.shape[0]
    n2 = y.shape[0]

```

```
    # Initialize
    np.random.seed(0)
    parameters = initialization(n0, n1, n2)

    # Training history
    train_loss = []
    train_acc = []

    for i in range(n_epochs):
        # 1. Forward
        activations = forward_propagation(X, parameters)
        A2 = activations['A2']

        # 2. Cost
        loss = log_loss(A2, y)
        train_loss.append(loss)

        # 3. Accuracy
        y_pred = predict(X, parameters)
        acc = accuracy_score(y.flatten(), y_pred.flatten())
        train_acc.append(acc)

        # 4. Backprop
        gradients = back_propagation(X, y, parameters, activations)

        # 5. Update
        parameters = update(gradients, parameters, learning_rate)

    return parameters, train_loss, train_acc
```

Let's train:

```
parameters, train_loss, train_acc = neural_network(
    X, y, n1=32, learning_rate=0.1, n_epochs=1000
)
```

What Happens Inside

At each epoch:

- The loss should **decrease** — the network is making smaller errors
- The accuracy should **increase** — the network is classifying more correctly
- The weights are being nudged, slowly, toward the right values

If the loss increases or oscillates wildly, the learning rate is probably too high.

After 1000 epochs with 32 hidden neurons and a learning rate of 0.1:

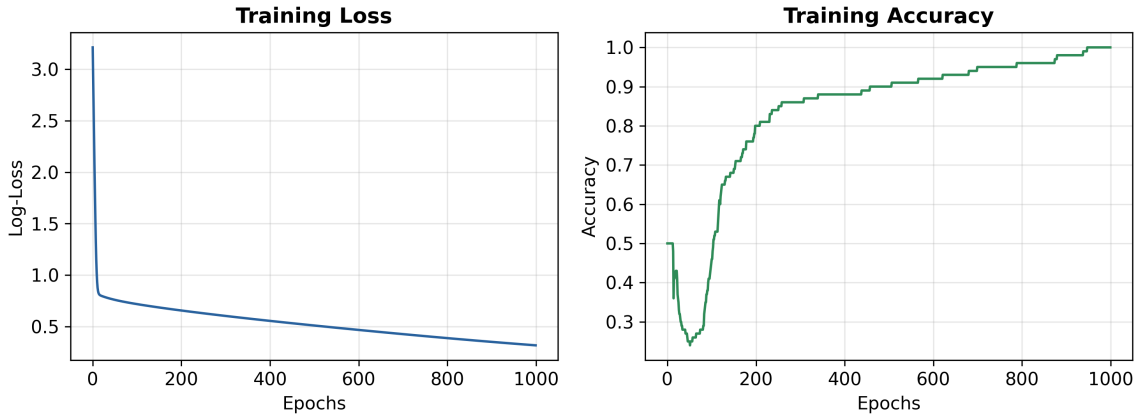


Figure 4: Training on `make_circles`: the loss drops sharply while accuracy climbs toward 100%.

The network learned.
From random weights to near-perfect classification — in a few seconds.

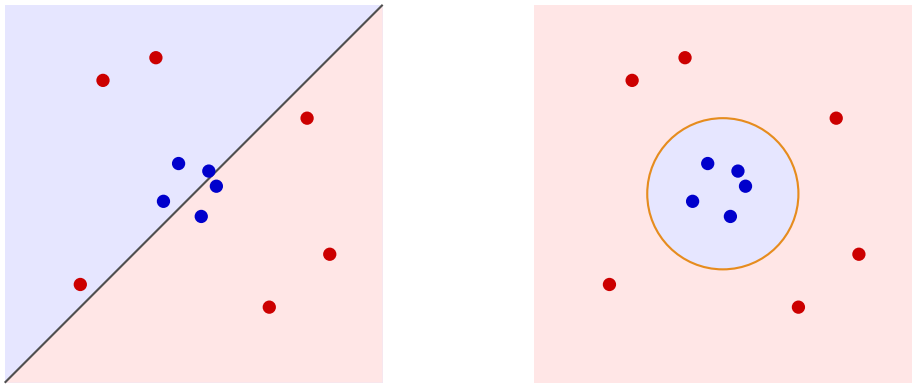
8 Decision Boundary — Seeing Intelligence

Let's visualize *what* the network learned.

The **decision boundary** is the set of points where the network is equally uncertain: $A_2 = 0.5$, meaning $Z_2 = 0$, meaning $W_2 \cdot A_1 + b_2 = 0$.

For a single neuron, this boundary is a straight line.

For a network? It's a **curve**.



1 neuron — straight line **32 neurons — curved boundary**

Figure 5: Left: a single neuron can only draw a straight line. Right: a hidden layer with many neurons can learn curved, nonlinear boundaries that wrap around the data.

How to Plot the Decision Boundary

1. Create a fine grid of points covering the input space
2. Run every grid point through the trained network
3. Color each point based on the network's prediction
4. The boundary appears where the color changes

```python
def plot_decision_boundary(X, y, parameters):
    x_min, x_max = X[0].min() - 1, X[0].max() + 1
    y_min, y_max = X[1].min() - 1, X[1].max() + 1
    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, 0.01),
        np.arange(y_min, y_max, 0.01)
    )
    grid = np.c_[xx.ravel(), yy.ravel()].T
    Z = predict(grid, parameters)
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
    plt.scatter(X[0], X[1], c=y, cmap='RdBu', edgecolors='k')
    plt.title('Decision Boundary')
    plt.show()
```

Run this on our trained model:

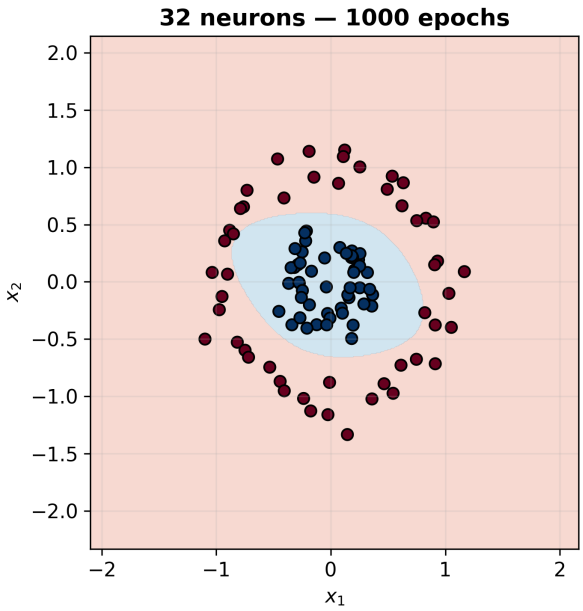


Figure 6: The trained network's decision boundary: a **nonlinear curve** that wraps around the inner cluster, separating toxic from non-toxic plants with near-perfect precision.

8.1 The Effect of Neuron Count

What happens when we change n_1?

Neurons	Behavior
1	Linear boundary (= logistic regression)
2	Slightly curved, not enough
4	Captures some nonlinearity
8	Clear circular-ish boundary
16	Smooth, accurate separation
32	Near-perfect classification

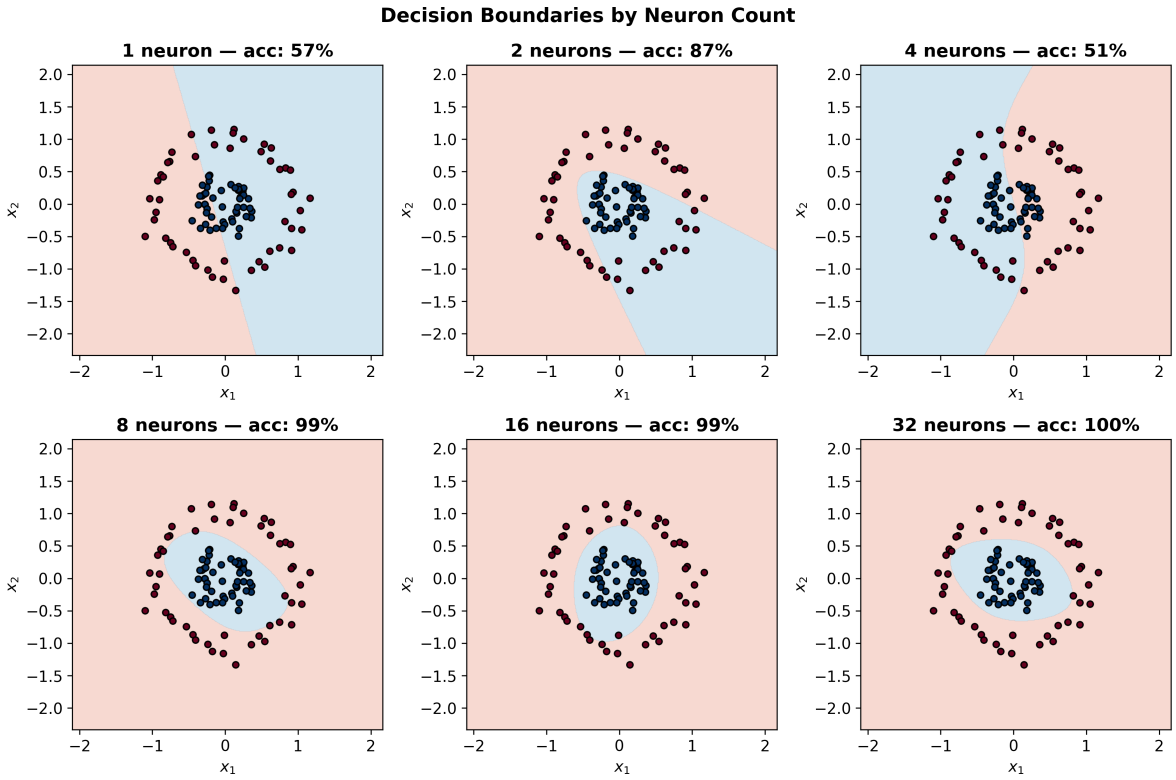


Figure 7: Decision boundaries for different hidden layer sizes. With 1 neuron, the boundary is a straight line (= logistic regression). As we add neurons, the network carves out increasingly complex, nonlinear boundaries.

More Neurons

Each neuron in the hidden layer contributes one "feature" to the representation. More neurons means the network can carve out more complex boundaries.
But be careful: too many neurons with too little data leads to **overfitting** — a problem we'll encounter very soon.

This is what intelligence looks like: the ability to draw the right boundary, automatically, from data alone.

9 Real Data — Cats vs Dogs

Toy datasets are great for learning. But let's see how our network handles real images.

We're going back to the **cats vs dogs** dataset from Episode IV.

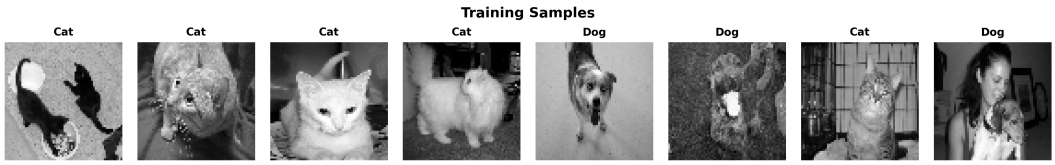


Figure 8: A few training samples from the cats vs dogs dataset — 64×64 grayscale images.

9.1 Data Preprocessing

Before feeding images into a neural network, we need to preprocess them:

1. **Load** the data from HDF5 files
2. **Reshape** each 64×64 image into a vector of 4096 pixels
3. **Normalize** pixel values to $[0, 1]$ by dividing by 255
4. **Transpose** to get shape `(n_features, n_samples)`

```python
from src.utilities import load_data

X_train, y_train, X_test, y_test = load_data()

# Reshape: (1000, 64, 64) -> (4096, 1000)
X_train_flat = X_train.reshape(X_train.shape[0], -1).T
X_test_flat = X_test.reshape(X_test.shape[0], -1).T

# Normalize
X_train_flat = X_train_flat / 255.
X_test_flat = X_test_flat / 255.

# Labels: shape (1, n_samples)
y_train = y_train.reshape(1, -1)
y_test = y_test.reshape(1, -1)
```

Data	Shape
`X_train_flat`	$(4096, 1000)$
`X_test_flat`	$(4096, 200)$
`y_train`	$(1, 1000)$
`y_test`	$(1, 200)$

Each image is now a column vector of 4096 numbers.

9.2 Training on Images

Here's the beautiful thing: we use the **exact same** `neural_network` function we wrote for toy circles — no changes needed.

```
parameters, train_loss, train_acc = neural_network(
    X_train_flat, y_train, n1=64,
    learning_rate=0.1, n_epochs=10000
)
```

Same Code, Different World

The network doesn't know it's looking at cat photos. It sees 4096 numbers and searches for the pattern that separates class 0 from class 1.
That's the power of abstraction: once the math is right, the data can be anything.

But we want to know more than just training performance. Does the network *generalize* — can it classify images it has never seen?

To find out, we modify the training loop to also track test loss and accuracy at each epoch:

```
def neural_network_with_test(X_train, y_train, X_test, y_test,
                             n1=32, learning_rate=0.1, n_epochs=1000):
    n0 = X_train.shape[0]
    n2 = y_train.shape[0]

    np.random.seed(0)
    parameters = initialization(n0, n1, n2)

    train_loss_hist, test_loss_hist = [], []
    train_acc_hist, test_acc_hist = [], []

    for i in range(n_epochs):
        # Forward + cost (train)
        act_train = forward_propagation(X_train, parameters)
        train_loss_hist.append(log_loss(act_train['A2'], y_train))

        # Forward + cost (test)
        act_test = forward_propagation(X_test, parameters)
        test_loss_hist.append(log_loss(act_test['A2'], y_test))

        # Accuracy
        train_acc_hist.append(
            accuracy_score(y_train.flatten(),
                           predict(X_train, parameters).flatten()))
        test_acc_hist.append(
            accuracy_score(y_test.flatten(),
                           predict(X_test, parameters).flatten()))

        # Backprop + update (train only)
        gradients = back_propagation(X_train, y_train,
                                     parameters, act_train)
        parameters = update(gradients, parameters, learning_rate)
```

```
33
34      return parameters, train_loss_hist, test_loss_hist, \
35             train_acc_hist, test_acc_hist
```

10 Overfitting — The First Warning

After training on the cats vs dogs dataset, you'll notice something disturbing:

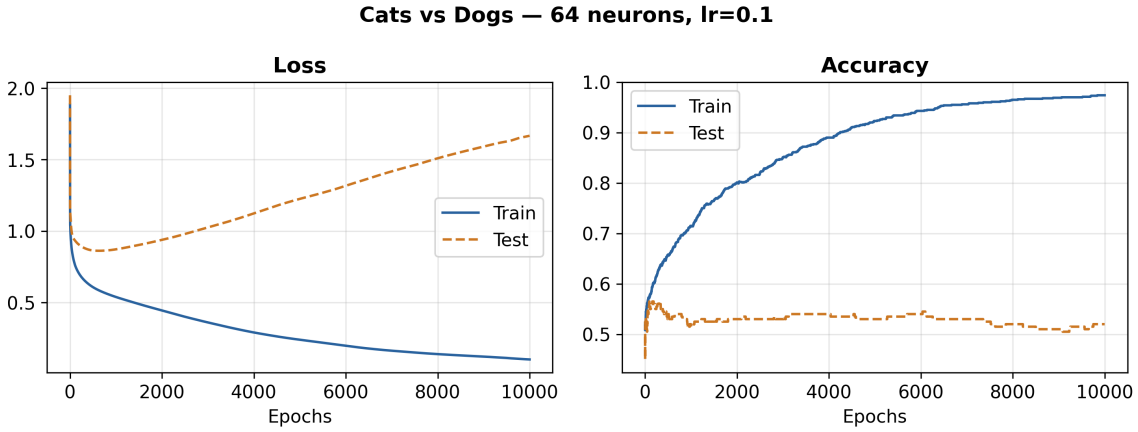


Figure 9: Train vs test performance on cats vs dogs. The training curves look great — but the test curves tell a different story. This is **overfitting**.

- Training loss keeps decreasing
- **Test loss starts increasing** after some point
- Training accuracy keeps climbing
- **Test accuracy plateaus** or even drops

Overfitting

When a model performs well on training data but poorly on unseen data, it's **overfitting**. The model isn't learning general patterns — it's **memorizing** the training examples. It's like studying for an exam by memorizing the answers instead of understanding the material.

Why does this happen?

1. **Too many parameters, too little data.** Our network with 64 hidden neurons has $4096 \times 64 + 64 + 64 \times 1 + 1 = 262{,}273$ parameters. But we only have 1000 training images.
2. **No regularization.** We haven't told the network to prefer simple solutions.
3. **No data augmentation.** We haven't artificially increased the training set size.

The Million-Dollar Question

When your model underperforms, ask yourself:

> **"Is this a problem with the data, the model, or the training?"**

Here, the answer is clear: not enough data for this many parameters.
A data scientist's job isn't just to build models — it's to **diagnose** why they fail and **decide** what to fix.

This is as far as our two-layer network can take us on this dataset.
But it's not a failure. It's a **lesson**.

11 What You've Built

Stop.

Before we move on — take a breath.

You started this series not knowing what a gradient was. Now you've just coded a neural network that classifies images of cats and dogs.

From scratch. Without a framework. Without anyone telling you what to copy-paste.

That's not nothing. That's rare.

Starting from nothing but a set of equations, you built:

Your Neural Network — From Scratch

1. **Initialization** — random parameters, controlled dimensions
2. **Forward propagation** — input to prediction in two matrix operations
3. **Cost function** — log-loss to measure errors
4. **Backpropagation** — six lines of gradient computation
5. **Update** — four subtractions (gradient descent)
6. **Training loop** — forward → cost → backward → update, repeat
7. **Prediction** — threshold at 0.5
8. **Visualization** — decision boundary plots

That's a complete neural network.

No framework. No library hiding the math. Just NumPy and understanding.

You tested it on toy data and got near-perfect classification. You tested it on real images and discovered overfitting. Both results matter.

> *Understanding why something fails is just as important as making it work.*

12 What's Next

Right now, our network has exactly two layers. We chose 32 neurons. We chose sigmoid activation.

But what if we wanted **three layers**? Or five? Or fifty?

Next Stop — Going Deeper

What if we could build a network with **any number of layers**, **any number of neurons per layer**, and **any architecture we want**?
That's not a hypothetical. That's the next episode.
We'll generalize everything we've done to L layers — and you'll see that the code barely changes.
Meet me in the next episode.

Deep Learning From Scratch

An open-source learning journey.

GitHub: https://github.com/Pchambet/Deep-Learning-from-Scratch

LinkedIn: https://www.linkedin.com/in/pierre-chambet/